

PATCH-BASED CONTRASTIVE LEARNING AND MEMORY CONSOLIDATION FOR ONLINE UNSUPERVISED CONTINUAL LEARNING

Cameron Taylor¹

cameron.taylor@gatech.edu

Vassilis Vassiliades²

v.vassiliades@cyens.org.cy

Constantine Dovrolis^{1,3}

constantine@gatech.edu

¹Georgia Institute of Technology
Atlanta, GA

²CYENS Centre of Excellence
Nicosia, Cyprus

³The Cyprus Institute
Nicosia, Cyprus

ABSTRACT

We focus on a relatively unexplored learning paradigm known as *Online Unsupervised Continual Learning* (O-UCL), where an agent receives a non-stationary, unlabeled data stream and progressively learns to identify an increasing number of classes. This paradigm is designed to model real-world applications where encountering novelty is the norm, such as exploring a terrain with several unknown and time-varying entities. Unlike prior work in unsupervised, continual, or online learning, O-UCL combines all three areas into a single challenging and realistic learning paradigm. In this setting, agents are frequently evaluated and must aim to maintain the best possible representation at any point of the data stream, rather than at the end of pre-specified offline tasks. The proposed approach, called **P**atch-based **C**ontrastive learning and **M**emory **C**onsolidation (PCMC), builds a compositional understanding of data by identifying and clustering patch-level features. Embeddings for these patch-level features are extracted with an encoder trained via patch-based contrastive learning. PCMC incorporates new data into its distribution while avoiding catastrophic forgetting, and it consolidates memory examples during “sleep” periods. We evaluate PCMC’s performance on streams created from the ImageNet and Places365 datasets. Additionally, we explore various versions of the PCMC algorithm and compare its performance against several existing methods and simple baselines. The code is publicly available [on Github](#).

1 INTRODUCTION

Imagine an agent navigating an unfamiliar environment with objects that have never appeared in its pre-training data. As the agent explores that environment, it must recognize new classes of objects. Furthermore, it must generalize from the observed data to other instances of the same class. Additionally, it should not forget previously learned classes, even if it does not encounter such instances often. This setup demands an efficient learning model where the agent operates in a streaming fashion, retaining only a minuscule fraction of the observed data due to storage or privacy constraints. We refer to this learning paradigm as *Online Unsupervised Continual Learning* or “O-UCL” for short.

The O-UCL learning paradigm must address the following three challenges: 1) The data stream is non-stationary, in the sense that the number of classes of objects in the stream increases with time; 2) the observed data is unlabeled, and so the agent needs to identify novel classes in real-time and without supervision; 3) the agent cannot store the observed data for future replay, necessitating online stream learning. While a simple solution might be to utilize a frozen encoder pretrained on the largest possible dataset, this is insufficient for applications in which the environment is inherently unknown and/or it includes novelty (e.g., people, animals or objects that were never previously seen). Instead, we consider a dynamic approach in which the encoder and the corresponding data representations are periodically adapted during short “sleep” periods.

For example consider the following hypothetical applications: an AI-powered drone that explores a new construction site or physical system, or a face recognition system that has to classify people it has never seen before with distinct identifiers. In addition to the three O-UCL challenges described above, such applications have one more common requirement: there is no boundary between training and inference. Instead, a successful O-UCL learner should both learn and perform its task during the data stream, exhibiting gradually better performance as it observes more data. In Figure 1, we demonstrate a toy scenario to exemplify O-UCL.

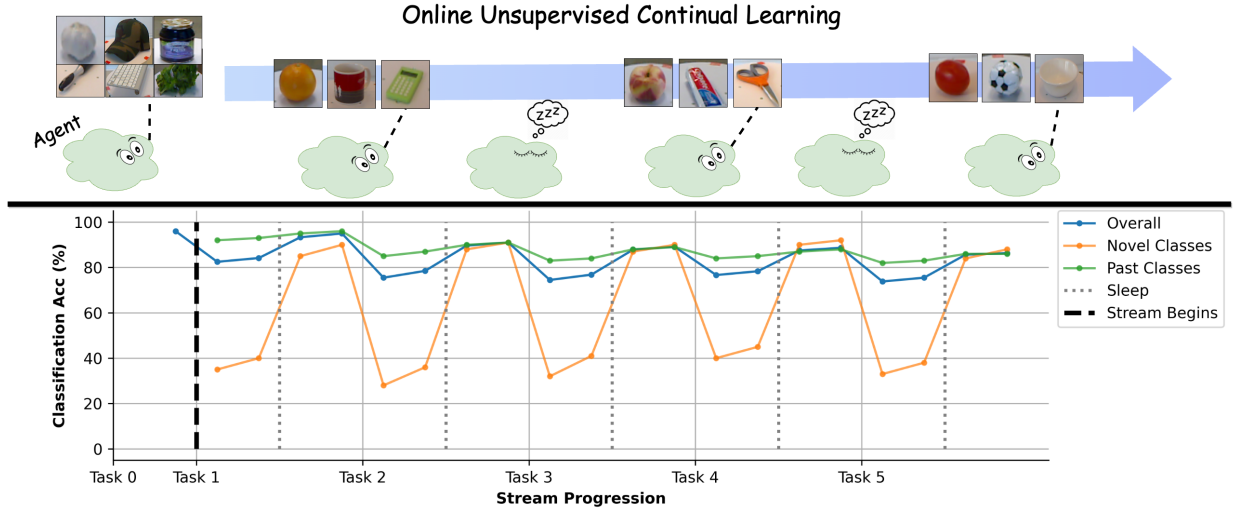


Figure 1: A toy example for an O-UCL scenario. After an offline initialization (task-0), the agent is presented with a stream of data consisting of images from various classes (say three new classes in each task). The agent is tasked with learning to identify both new and previously known classes, without forgetting classes that no longer appear in the stream. The performance is evaluated frequently during the stream, to monitor how the agent learns over time. During sleep periods, the agent retrain its encoder and adapts its stored representations. Note that a small number of labeled examples are given to the classifier only during inference – no labeled examples are available for representation learning during the stream.

A similar problem to O-UCL was first introduced in (Smith et al., 2021), referred to as “Unsupervised Progressive Learning.” That work also proposed a method called “Self-Taught Associative Memories” or STAM. The method we propose here, referred to as **Patch-based Contrastive learning and Memory Consolidation** or “PCMC,” utilizes some techniques from STAM but it also introduces several new ideas. PCMC operates in a cycle of “wake” and “sleep” periods. The model identifies and clusters incoming stream data during each wake period, while it retrain the encoder and consolidates data representations during each sleep period. PCMC utilizes a novel patch-based contrastive learning encoder along with online clustering and novelty detection techniques. We evaluate PCMC’s performance against several baselines on challenging natural image datasets such as ImageNet (Deng et al., 2009) and Places365 (Zhou et al., 2017).

2 PCMC METHOD

The key component of PCMC is a growing set of cluster centroids that represent distinct data features. To learn useful centroids, we utilize an encoder $F_\phi(\cdot)$, which is a deep neural network with parameters ϕ that generates similar embeddings for similar inputs and dissimilar embeddings for dissimilar inputs (contrastive learning). As the distribution of the stream changes over time, the encoder must adapt to changes in the data distribution (e.g., novel classes) but also to avoid concept drift and catastrophic forgetting. PCMC accomplishes this by adopting a wake-sleep cycle. During a wake period, the model performs novelty detection (i.e., creation of new clusters) and it also adapts the centroids of previously known clusters with new data. During a sleep period, the encoder is retrained and also the representations of the learned centroids are updated.

Critically, instead of clustering entire input examples (images in this paper), PCMC breaks each incoming example into smaller patches, and maps each patch to a cluster that represents a distinct feature of the data. The use of patches is important for two reasons. The first is that as the data distribution changes, we want to leverage potential forward transfer for shared features across classes (e.g., fire truck tires and ambulance tires) at the level of patches. Secondly, clustering similar inputs is more challenging at the level of the entire input compared to the patch level. This is because the variance of a patch-level feature (e.g., a dog’s nose) is typically much smaller than the variance of the entire input (e.g., an image of a dog). By learning features at the patch level, the model can build a compositional understanding of each class, based on the class-informative features it includes.

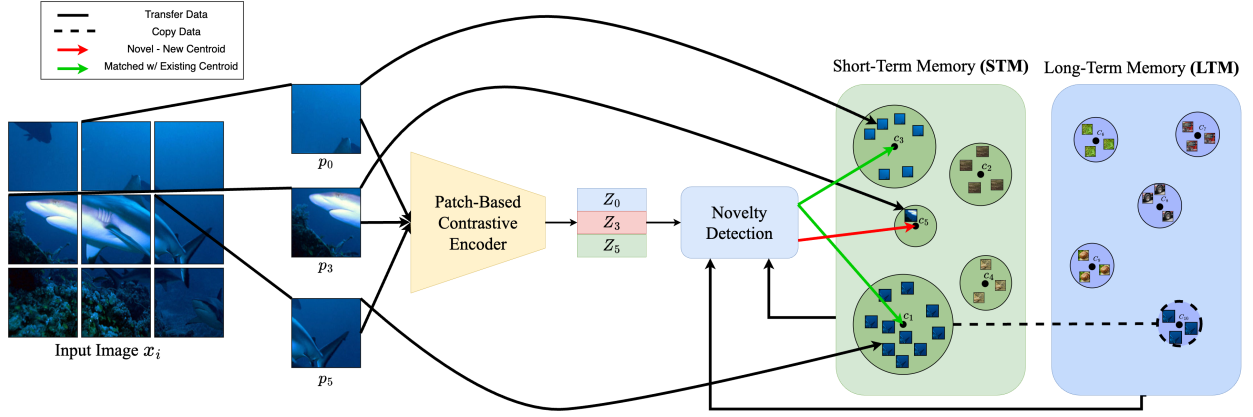


Figure 2: This figure summarizes the wake period of PCMC. The input x_i is broken up into patches, and the encoder F_{ϕ_s} generates an embedding for each patch. A patch embedding is compared with existing centroids to perform novelty detection. If the embedding is far from any stored centroid, a new cluster is created in Short-Term Memory. Otherwise, that patch is mapped to its nearest centroid – and the location of the latter is updated. When a cluster accumulates several (θ) patches, it is copied from Short-Term Memory to Long-Term Memory so that it is never forgotten.

2.1 PATCH-BASED CONTRASTIVE LEARNING

To learn an encoder with the previous properties, we propose a patch-based modification of typical instance-based contrastive learning. Given an input image x , we extract patches $\mathcal{P} = \{\mathbf{p}_i\}_{i=1}^N$ and apply a set of augmentations (see Section 3) to each patch to generate $\mathbf{p}_{i,1}$ and $\mathbf{p}_{i,2}$. To maximize the number of training samples extracted from each image, we extract highly overlapping patches using a stride that is smaller than the patch size. We ensure that highly overlapping patches are always in separate batches. We also experimented with the SupCon loss from (Khosla et al., 2020), where any highly overlapping patches would be given the same label. However, we observed better results when simply splitting overlapping patches into different batches. In this work, we utilize the loss from (Chen et al., 2020a), but our framework is compatible with any instance-based contrastive learning loss. The loss function for a single positive pair of patch embeddings $(\mathbf{z}_i, \mathbf{z}_j)$ is given by:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)} \quad (1)$$

where N is the total number of patches in the batch and $\text{sim}(\cdot, \cdot)$ is the cosine similarity.

2.2 WAKE PHASE

Centroid Learning: When a new input $\mathbf{x}$ is received, the model extracts patches of $\mathbf{x}$ based on a given patch size and stride. These patches are then encoded with F_{ϕ} . The patch embeddings are then processed sequentially using an online clustering algorithm: given a patch $\mathbf{p}$, we find the most similar cluster by mapping the embedding of that patch $\mathbf{z}$ with the nearest cluster centroid in the set of existing centroids $\mathcal{C}$ as

$$\mathbf{c}_j = \arg \min_{\mathbf{c} \in \mathcal{C}} d(\mathbf{z}, \mathbf{c}) \quad (2)$$

where $d(\cdot, \cdot)$ can be any distance metric; we use cosine distance. If the nearest cluster is within the *novelty detection* threshold (see later in this section), we expect that the cluster consists of patches that have high similarity with $\mathbf{p}$ and assign $\mathbf{p}$ to cluster $\mathbf{c}_j$. When a patch is assigned to a cluster, its embedding updates the centroid as follows,

$$\mathbf{c}_j = (1 - \alpha)\mathbf{c}_j + \alpha \mathbf{z} \quad (3)$$

and the raw patch $\mathbf{p}$ is temporarily stored with the selected centroid in an auxiliary memory $\mathcal{M}_j$. The centroid learning rate α controls how much influence an individual input has on the centroid, and it can be set as a fixed value or it can decay as the size of the cluster increases; we do the former.

Novelty Detection: If the distance between $\mathbf{z}$ and its nearest cluster centroid $\mathbf{c}_j$ is greater than the novelty detection threshold $\hat{D}$, a new cluster is created and $\mathbf{z}$ becomes the centroid of the new cluster. The raw patch $\mathbf{p}$ is also temporarily

stored in the new cluster’s memory M . Because the data distribution changes over time, the threshold $\hat{D}$ should also be dynamically updated. We estimate $\hat{D}$ in an online manner by maintaining a distance distribution between recent patch embeddings z and their nearest centroid c_j , using a sliding window over recently observed samples. The novelty detection threshold is defined as a high percentile (β) of this distance distribution.

Short & Long Term Memory: The model partitions the learned cluster centroids into two groups: the short-term memory (STM) and the long-term memory (LTM). The STM maintains centroids that were recently initialized. It has a fixed size and uses a least-recently used replacement policy to create space for new centroids, once it has reached capacity. The purpose of the STM is to temporarily store clusters until they have been matched multiple times (θ), allowing the model to be more certain that this cluster represents an actual feature of the data, and not an outlier or noise. Whenever a centroid c_j in the STM has reached θ matches, it is copied to the LTM along with its memory $\mathcal{M}_j$. The STM centroid c_j is marked as unusable until it is evicted or the model goes to sleep (more details in 2.3). The purpose of the LTM is to permanently store useful features that have been observed multiple times in the stream. Those cluster centroids are frozen to prevent concept drift and catastrophic forgetting, while the corresponding cluster memory is utilized to retrain the encoder during sleep periods, as explained next.

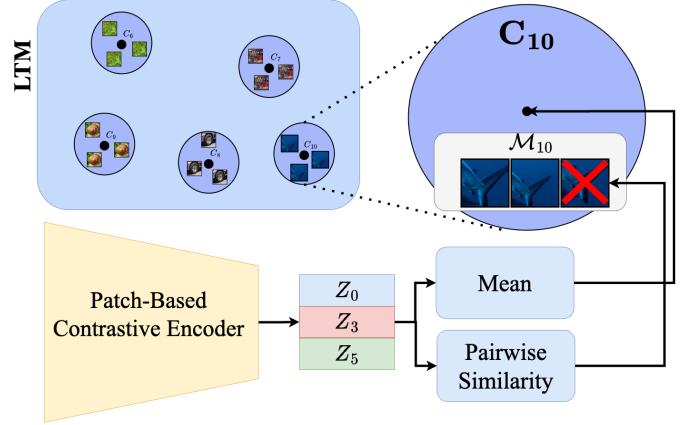


Figure 3: The memory consolidation process during the sleep phase of the PCMC algorithm. Each centroid in the model’s LTM is recomputed using the updated contrastive encoder. Very similar examples stored in the centroid’s memory are pruned.

2.3 SLEEP PHASE

Periodically, PCMC goes to sleep to update the encoder based on the recently seen data since the last sleep period. Additionally, sleep is used to consolidate the training patches stored in LTM. The sleep phase consists of two stages. The first is the encoder retraining stage, which utilizes the stored patch examples to fine-tune the encoder’s weights. The second is the centroid update stage, which updates the centroids’ positions and stored examples, avoiding concept drift and improving efficiency.

Encoder Retraining: PCMC creates a new dataset from the set of patch examples stored in STM and LTM, which we call X_{sleep} . The STM data is meant to provide new information from the most recently created clusters, while the LTM data serves to remember previously learned centroids to avoid catastrophic forgetting. The encoder $F_{\phi_{s-1}}(\cdot)$, with weights ϕ_{s-1} after $s - 1$ sleep cycles, is then updated using the contrastive learning method of Section 2.1.

Centroid Updates: After the encoder is updated, PCMC updates the centroids in both STM and LTM to avoid suffering from concept drift. The model re-embeds each centroid using K examples from that centroid’s memory $\mathcal{M}_j$ as

$$c_j = \frac{1}{K} \sum_{i=1}^K F_{\phi}(\mathcal{M}_{j,i}) \quad (4)$$

In the experiments of this paper, PCMC utilizes a single example ($K = 1$) as opposed to the average of all stored examples. The single example is chosen as the first example added, which is also the example used to originally create the centroid. After that point, the STM example memories M are erased, and the counter of STM matches is reset.

Memory Consolidation: Once the centroid positions have been updated, PCMC utilizes a targeted forgetting strategy to help prune some of the more redundant examples stored in LTM. At this stage, each centroid c_j is assigned a probability P_{prune} of pruning a single example from its own memory examples $\mathcal{M}_j$. Specifically, the probability scales with the number of examples stored and is given by

$$P_{\text{prune}}(\mathcal{M}_j) = \left(\frac{|\mathcal{M}_j| - M_{\min}}{M - M_{\min}} \right)^k \quad (5)$$

where $|\mathcal{M}_j|$ is the number of examples stored with the j 'th centroid, M_{min} is a hyper-parameter controlling the minimum number of examples a centroid can have, and k is a scaling factor that impacts the frequency of pruning. This equation allows us to select a pruning probability that decreases from 1 to 0 based on a desired maximum (M) and minimum (M_{min}) number of stored examples, while k controls how aggressive the pruning is as $\mathcal{M}_j$ gets closer to M_{min} . In Section 4.2 we showcase how changing M , or not performing this consolidation step, impacts performance and also the overall memory requirements of the PCMC model. Further experiments with k and M_{min} are given in Appendix-C.2

2.4 INITIALIZATION

To bootstrap the initial novelty detection values and encoder weights, the model starts with an initialization task T_0 that consists of unlabeled images. The data $X(T_0)$ is available upfront. The model trains on the data using the approach described in 2.1. After training, we utilize k-means to cluster a randomly sampled subset of $X(T_0)$ into Δ_0 clusters, and initialize both STM and LTM with these centroids. In the LTM, each centroid also stores M_{init} raw patches that are members of that centroid's cluster so they can be used later during the sleep phases. We also initialize $\hat{D}$ by sampling a small set of distances between random patches from $X(T_0)$ and the nearest centroid in the initialized STM. The initial novelty detection threshold $\hat{D}$ is set to the β 'th percentile of that distance distribution

3 EXPERIMENTAL SETUP

In this section, we describe the datasets, training details, and evaluation methods used in our experiments.

3.1 DATASETS AND STREAMS

We create streams from two different datasets: ImageNet Deng et al. (2009) and Places365 Zhou et al. (2017). The ImageNet-derived stream focuses on object classification, while the Places365 stream focuses on scene classification. Both streams consist of 40 classes and are referred to as ImageNet-40 and Places365-40, respectively. Each stream comprises 16 tasks: the first task (T0) is offline and contains 10 classes, while the remaining 15 tasks each contain two classes, forming the stream. We resize each image to 120×120 pixels and normalize them. Tables containing all hyperparameters for PCMC and baselines can be found in Appendix-A.2 and Appendix-B.

3.2 TRAINING DETAILS

Initialization: We use a ResNet18 He et al. (2016) as the encoder backbone and a two-layer MLP with a hidden size of 512 for the projection head during contrastive training. The encoder is trained for 500 epochs using the SGD optimizer with an initial learning rate of 0.6, a weight decay of $1e-5$, a momentum of 0.9, and a cosine annealing learning rate schedule. For the initial novelty detection threshold estimate and initial STM/LTM centroids, we utilize a subset of the T0 data as described in Section 2.4.

Sleep Cycle: In our primary experiments, PCMC goes to sleep at fixed intervals during each task. In 4.4, we consider the frequency and timing of sleep cycles, demonstrating that timing is not critical to overall performance. During each sleep cycle, we train the ResNet18 backbone for 300 epochs using the same optimization hyperparameters as during initialization. Since the sleep dataset stores patches without reference to the original image, each training batch inconstrative learning consists of independent patches, not entire images.

Augmentations: We use a similar set of augmentations to Chen et al. (2020a) – but with some tweaks to optimize for our patch-based approach. We reduce the magnitude of **ColorJitter** and remove **GaussianBlur** entirely to emphasize color and texture in defining visual similarity. Additionally, we modify **RandomCropandResize** by applying the crop only to the first augmented version, while keeping the range of possible crop sizes for the second version closer to the full size. This encourages invariance to small changes in translation and scale, while avoiding overly drastic transformations.

3.3 EVALUATION

For classification and clustering tasks, we use an approach similar to Smith et al. (2021), as explained next.

Classification with PCMC: To perform classification, PCMC takes a small set of labeled images X_L and breaks each image into patches. For a patch p from an image of class k , each centroid gets an association distribution g_c based on

$$g_c(k) = \frac{\sum_{\mathbf{p} \in X_L(k)} D(\mathbf{p}, \mathbf{c}_j)}{\sum_{k' \in L(t)} \sum_{\mathbf{p} \in X_L(k')} D(\mathbf{p}, \mathbf{c}_j)}, \quad k = 1 \dots L(t), \quad (6)$$

where $D(\mathbf{p}, \mathbf{c}_j)$ is the distance between patch p and its nearest centroid c_j . Centroids that are strongly associated with one class are considered “class informative”. PCMC then breaks down test images X_T into patches, finds the most similar centroid c_j for each patch, and if that centroid is class informative, it becomes an eligible voter. Eligible voters cast a vote for class k^* based on

$$k^* = \arg \max_{k \in L(t)} g_{c_j}(k) \quad (7)$$

with the vote’s magnitude equal to $g_{c_j}(k^*)$. The total vote for class k is the sum of votes for k , and the final class prediction is the highest voted class.

Clustering with PCMC: To perform tasks that do not require any labels, such as offline clustering, we use PCMC features. For each patch of input $\mathbf{x}$, we compute the nearest LTM centroid. The set of all such centroids across all patches of $\mathbf{x}$ is denoted by $\Phi(\mathbf{x})$. Given two inputs $\mathbf{x}$ and $\mathbf{y}$, their pairwise distance is the Jaccard distance of $\Phi(\mathbf{x})$ and $\Phi(\mathbf{y})$.

To perform clustering on a given set of inputs and a target number of clusters, we apply a standard spectral clustering algorithm on all pairwise distances $\Phi(\mathbf{x})$ and $\Phi(\mathbf{y})$. Other clustering algorithms can also be used.

3.4 BASELINES

Whole-Image Baseline: This baseline exemplifies the wake/sleep cycle used with PCMC, but without the patch-based approach. It uses contrastive learning and it stores entire images (instead of patches) over time. Specifically, the model randomly selects and stores M_{init} images from T0 and M images from each task, adding them to a long-term memory (LTM). It periodically goes to sleep and retrains the encoder using a contrastive objective on the LTM images. The M and M_{init} values are slightly more than the actual memory usage of PCMC to ensure a fair comparison. The whole-image baseline trains for 500 epochs during T0 and 300 epochs for each subsequent sleep cycle. The encoder backbone is a ResNet18 [He et al. \(2016\)](#) with a two-layer MLP for the projection head, using the SimCLR loss and augmentations.

SCALE [Yu et al. \(2022\)](#) : We modify SCALE to include pretraining during T0, using its contrastive loss. The memory bank size is fixed, equivalent to the final memory utilization of PCMC. We use the same hyperparameters with SCALE for TinyImageNet [Le & Yang \(2015\)](#), with slight modifications to batch size and learning rate for the O-UCL setting. We also explored other hyperparameter values in Appendix-B.1 to ensure a fair comparison but found no substantial improvements. SCALE was not designed to include pretraining, and so we also experimented with a version that learns from T0 in an online streaming manner, but its performance was poor relative to PCMC.

STAM [Smith et al. \(2021\)](#) : We use the [original STAM code](#) with minor changes for computational efficiency. STAM was not designed to include a pretraining phase, and so the T0 data is presented as an additional streaming task, allowing STAM to use a portion of that data for initialization. We use similar hyperparameters to those in the original STAM paper for CIFAR10 [Krizhevsky et al. \(2009\)](#), but scaled up for larger resolution images. The full set of hyperparameters is in Appendix-B.3.

4 EXPERIMENTAL RESULTS

4.1 COMPARISONS WITH BASELINES

Figure 4 shows classification and clustering performance of PCMC compared to several baselines. For ImageNet-40, the results are shown in the left column, with the first row representing classification accuracy and the second row showing clustering purity. Similarly, the Places365-40 results are shown in the right column. The performance of PCMC does not degrade significantly over time as new classes are added, highlighting its ability to incorporate new knowledge without major catastrophic forgetting. In Appendix C, we also examine the performance of PCMC broken down by “novel” and “past” classes within each task. This demonstrates how the model quickly improves its understanding of new classes, with a significant boost after each sleep phase. Additionally, we examine forgetting

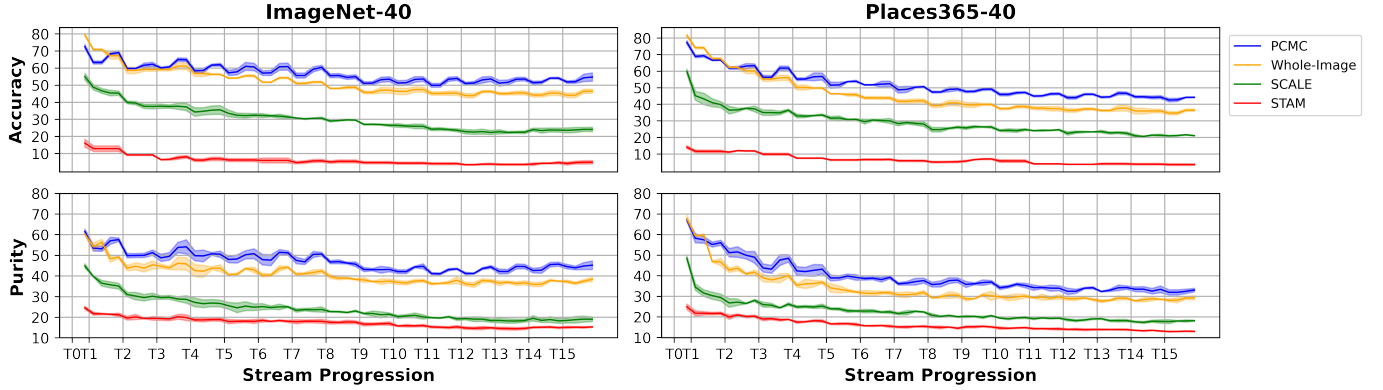


Figure 4: Classification and clustering performance comparisons between PCMC and baselines on the ImageNet-40 and Places365-40 streams. In both streams, the initial task T0 contains 10 classes, and each of the subsequent 15 tasks contains 2 classes each. Each task contains four evaluation points distributed evenly throughout the task, focusing on all classes seen so far. For the classification tasks, we use 100 labeled examples per class and 100 test examples per class. We emphasize that these labeled examples are not used for representation learning during the stream – they are only used to identify class-informative centroids. Average results over three independent seeded trials are shown, with error measured as $\pm$ one standard deviation.

over time, both pre- and post-sleep, showing how the sleep cycle helps the model disentangle class representations and improve performance on both past and novel classes compared to early evaluations during each task.

4.2 IMPACT OF M ON PERFORMANCE

Table 1 presents an evaluation of PCMC with three different values for M , along with the total amount of memory used at the end of the stream

Table 1: Comparison of classification performance versus memory usage for PCMC and PCMC-NC without memory consolidation. Results are calculated over three independent seeded trials. $|LTM|$ is the maximum memory (in terms of images) utilized across all three trials, and accuracy is the average across all evaluations $\pm$ one standard deviation.

	M	ImageNet-40		Places365-40	
		$ LTM $	Accuracy	$ LTM $	Accuracy
PCMC	10	4647	54.7 ± 0.51	4383	51.6 ± 0.63
	20	8819	59.7 ± 1.34	8660	55.7 ± 0.98
	30	13250	59.9 ± 0.95	14618	55.5 ± 0.88
PCMC-NC	10	6966	56.3 ± 0.84	6040	52.3 ± 0.40
	20	12715	60.0 ± 1.21	12800	55.8 ± 0.97
	30	18287	60.0 ± 0.54	18983	56.3 ± 0.64

(in terms of whole images) and the average overall performance. The first three rows represent the actual PCMC algorithm, including the memory consolidation step, while the final three rows represent a PCMC ablation, without the memory consolidation step (PCMC-NC). For each value of M , the consolidation algorithm saves approximately 30% of memory with minimal impact on overall performance. For this experiment, we consider a forgetting factor $k = 2$, a minimum centroid capacity of $M_{min} = 5$, and $M_{init} = M$. In Appendix C.2, we explore more values of k and M_{min} , noting that more aggressive example pruning has a negative impact on performance.

4.3 NOVEL VS PAST CLASS PERFORMANCE

Figure 5 shows the performance of PCMC on ImageNet-40 and Places365-40, broken down into “novel” classes, “past” classes, and “overall”, similar to Figure 1. This plot helps to understand how PCMC adapts to new classes while maintaining performance on previously seen classes.

We observe that in both figures, the model improves its performance on novel classes throughout the task. The extent of improvement varies depending on the difficulty of the new classes. If the model struggles to adapt to new classes, the performance on past classes drops when we enter a new task, as the previously “novel” classes now become “past” classes. However, if the model adapts well, the overall performance can increase slightly, maintaining consistent performance throughout the stream.

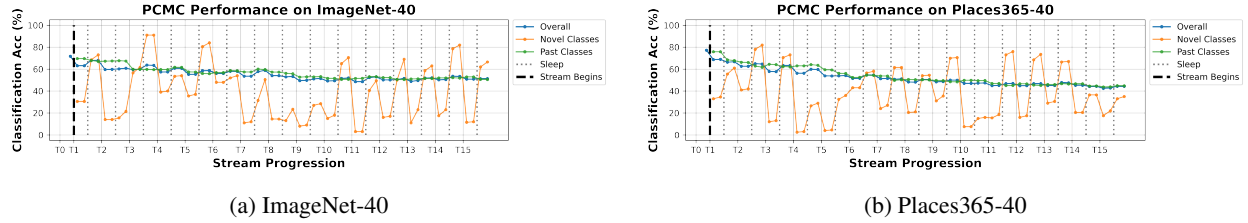


Figure 5: PCMC classification performance breakdown for a specific trial on the ImageNet-40 and Places365-40 streams. The orange curve represents the performance on the novel classes, the green curve represents performance on the past (previously observed) classes, and the blue curve represents the overall performance. The vertical grey dashed lines represent sleep cycles.

4.4 ABLATIONS

Sleep Timing: Figure 6a compares the classification performance of PCMC with different sleep cycle timings. The “sleep-middle” curve represents the traditional PCMC, where the model goes to sleep at fixed intervals in the middle of each task. The “sleep-less” curve represents the model sleeping in the middle of every other task, and the “sleep-end” curve represents the model sleeping at the end of each task. The most significant difference is observed in the “no-sleep” version of PCMC, while the “sleep-end” and “sleep-less” versions are closer to the primary PCMC approach. Interestingly, sleeping less is more beneficial than sleeping at the end of the task. We assume this is because the mid-task sleep cycle allows the model to integrate some understanding of the new distribution into the encoder, which can then be used to learn better centroids in the later half of the task. Future work could explore a version of PCMC that detects shifts in the data distribution (possibly via its existing novelty detection mechanisms) and goes to sleep after collecting a sufficient number of novel examples.

Patch Size Comparison: Figure 6b shows an ablation study comparing PCMC with different patch sizes. The key takeaway is that patch sizes that are either too small or too large significantly harm performance. However, several intermediate patch sizes yield similar performance. In our primary experiments, we chose a patch size of 60 over 90 because it allows us to save about 30% of the total memory budget. Even with increased memory allowance, the smaller patch size of 40 and the whole-image version of PCMC do not compare. This comparison highlights the benefits of patches both in terms of memory efficiency and in incorporating new data into the model’s learned representations.

Comparison with Upper & Lower Bounds: Figure 6c demonstrates the performance of PCMC with a frozen encoder that is pretrained in an offline fashion with all classes from the entire stream (PCMC-Pre-All), a frozen encoder that is pretrained with self-supervision only on the T0 data (PCMC-Fixed), and a series of PCMC models with dynamic encoders that use progressively more memory for storing raw examples. We also include a “Supervised Offline” upper bound that is based on a ResNet18 encoder with the pretrained ImageNet-1k weights and a k-NN classifier. The Supervised Offline and PCMC-Pre-All versions are meant to act as upper bounds, when the models operate with “ideal” encoders. Note that as the number of classes increases, even this ideal encoder-based model suffers from a decrease in performance due to increased task complexity. This may seem surprising given that the pretrained encoder in this case was trained on the entire ImageNet-1k dataset, with labels. However, the use of PCMC’s classification algorithm and the limited number of labeled examples per class make this task more challenging than typical 40-way classification. As we increase the amount of memory that PCMC utilizes, we see that the gap in overall performance closes and it is much closer to the Pre-All upper-bound. However, the gap with the supervised offline approach is still large.

Alternative Encoder Architectures: We also compare PCMC, SCALE, and the Whole-Image baseline with various encoder sizes. Table 2 showcases the performance of the three approaches utilizing neural network-based encoders. The performance is shown in terms of the average classification accuracy and clustering purity throughout all evaluations in the stream. The three architectures considered are ResNet18, ResNet34, and ResNet50, with their parameter counts shown in the first column of the table. When comparing approaches that utilize the same backbone, PCMC’s performance is the best. Additionally, for both classification and clustering, PCMC outperforms all versions of the Whole-Image baseline and SCALE with just the ResNet18 backbone. With SCALE, we see a decrease in performance as the encoder size increases, likely due to the limited number of training iterations in SCALE. This makes learning more difficult with a larger number of parameters and causes performance to drop throughout the stream.

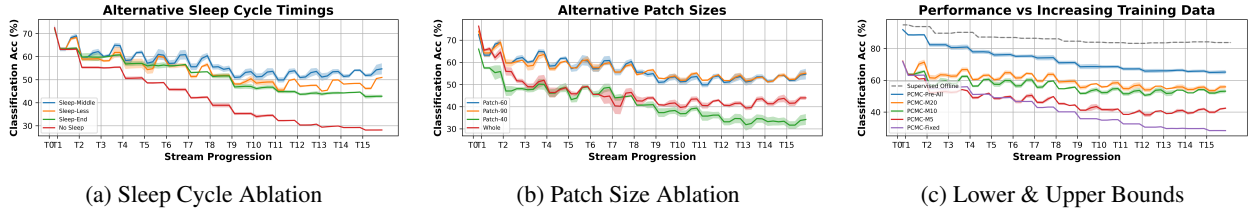


Figure 6: PCMC ablations exploring the various modeling choices and performance bounds. a) explores the impact of the timing of the sleep cycle on the overall performance, b) examines the impact of patch size and also potential benefits of using patches at all, while c) compares PCMC with an upper and lower bound in terms of the quality of the encoder. Each experiment is the average over three independent seeded trials with error measured as $\pm$ one standard deviation.

5 RELATED WORK

Online Unsupervised Continual Learning: Variations of the O-UCL problem have been studied under different names. (Smith et al., 2021) was the first to introduce a similar problem called “Unsupervised Progressive Learning” (UPL). While O-UCL and UPL share some similar characteristics, there are also important differences. UPL does not include a pretraining phase with some initial data. Also, UPL does not allow the model to save any raw examples (images or patches) during the stream. In the case of O-UCL, we allow for some data to be available upfront for pretraining, and include the corresponding classes in the evaluation set. SCALE (Yu et al., 2022) and DAA (Michel et al., 2023) address a problem that is similar to O-UCL but they allow for storage of entire images and do not include an initialization task. SCALE combines mixture replay with contrastive learning to learn from an unlabeled stream with a changing distribution, while maintaining a fixed memory of examples to avoid forgetting. DAA approaches the problem similarly to SCALE but focuses on novel augmentation strategies applied to stream and replay images. Specifically, DAA utilizes MixUp, CutMix, and Style transfer to combine stored examples with new examples. Unfortunately, we were not able to reproduce the DAA method and its original implementation is not yet publicly available. OUPN (Ren et al., 2021) studies another similar setting but focuses on understanding the current distribution rather than avoiding catastrophic forgetting over time. It learns an online mixture of Gaussians similar to (Rao et al., 2019) but instead of just using a reconstruction loss, the model contrasts cluster (mixture component) assignments.

Table 2: Comparison of PCMC, SCALE, and Whole-Image Baseline on ImageNet-40 with several different backbone architectures. Results show average accuracy and purity across all evaluations throughout the stream, consisting of three independently seeded trials showing mean $\pm$ one standard deviation.

	Params	Accuracy	Purity
PCMC-ResNet18	11.7 M	56.6 ± 1.3	47.2 ± 0.2
PCMC-ResNet34	21.8 M	57.9 ± 1.6	47.6 ± 1.0
PCMC-ResNet50	25.6 M	58.6 ± 0.8	48.3 ± 0.3
SCALE-ResNet18	11.7 M	30.4 ± 0.8	24.1 ± 0.7
SCALE-ResNet34	21.8 M	16.3 ± 0.6	18.0 ± 0.5
SCALE-ResNet50	25.6 M	22.9 ± 1.3	20.6 ± 0.5
Whole-ResNet18	11.7 M	51.6 ± 0.6	40.9 ± 0.4
Whole-ResNet34	21.8 M	53.9 ± 0.3	43.3 ± 0.3
Whole-ResNet50	25.6 M	55.3 ± 0.5	44.9 ± 1.4

Continual Learning: Continual learning learns a sequence of different tasks over time. Typically, it is assumed that each task has a fixed data distribution (Hsu et al., 2018). Some approaches focus on regularizing the “important” weights learned in previous tasks (Kirkpatrick et al., 2017; Li & Hoiem, 2017; Rannen et al., 2017; Aljundi et al., 2018; 2019a; Lopez-Paz & Ranzato, 2017). Others focus on storing representative examples or representations and replaying them during future tasks (Aljundi et al., 2019b; Buzzega et al., 2020; Rebuffi et al., 2017; Rolnick et al., 2019; Shin et al., 2017). More recently, some works have focused on improving the efficiency of replay memories (Brignac et al., 2023; Hurtado et al., 2023; Bai et al., 2023). Some methods focus on the supervised online continual learning setting (Lee et al., 2023; Hayes et al., 2020; Hayes & Kanan, 2020), while others focus on memory or compute-constrained environments (Ghunaim et al., 2023; Demosthenous & Vassiliades, 2021; Hayes & Kanan, 2022; Fini et al., 2020; Harun et al., 2023). These works may have slightly different learning criteria and objectives but they all differ substantially from our work because they rely on labels.

Our work does not assume knowledge of task boundaries, which is similar to “Task-Free Continual Learning” (TFCL) (Aljundi et al., 2019a) but that work does not consider learning from a stream.

The streaming aspect of O-UCL is also similar to “Online Continual Learning” (OCL) (Aljundi et al., 2019b) but the data in O-UCL is unlabeled. Several other works under the umbrella of continual learning have looked at similar problems but they do not consider all challenges at the same time. For instance, some works examine the unsupervised continual learning problem with a focus on learning generative models in a continual but offline manner (Rao et al., 2019; Ye & Bors, 2020; Lee et al., 2020; Ramapuram et al., 2020; Achille et al., 2018). Other works focus on self-supervision for good representations or use a contrastive loss to contrast between new and old samples across tasks (Morawiecki et al., 2022; Ni et al., 2021; Zhang et al., 2020; Cha et al., 2021; Li et al., 2022b; Madaan et al., 2021; Fini et al., 2022; Lin et al., 2022).

Self-Supervised Learning: Self-supervised learning focuses on learning to extract useful representations from unlabeled data. Some of the earlier computer vision self-supervised learning methods involved utilizing pretext tasks such as inpainting (Pathak et al., 2016), colorization (Deshpande et al., 2015; Larsson et al., 2016; Zhang et al., 2016), denoising (Vincent et al., 2008), and making predictions about rotation or relative position (Gidaris et al., 2018). More recently, instance-based approaches that contrast augmented versions of different images have been proposed (Chen et al., 2020b;a; Chen & He, 2021; He et al., 2020; Grill et al., 2020). Expanding on this idea, cluster-based approaches that contrast cluster assignments of augmented versions of different images have become much more popular (Li et al., 2020; 2022a; Caron et al., 2018; 2020). Our work is designed to utilize an instance-based contrastive objective for training the encoder, but the O-UCL framework is agnostic to the specific method.

6 CONCLUSIONS

This work makes two contributions. First, we present the Online Unsupervised Continual Learning problem and make the case that it captures many real-world applications, even though it is also a much harder problem to define mathematically and solve in practice. The second contribution is a method (PCMC) that attempts to solve the O-UCL problem.

PCMC is designed to operate effectively in complex, real-world environments, learning from non-stationary and single-pass data streams without the need for external supervision or predefined knowledge. PCMC’s dynamic encoder and patch-based contrastive learning allow for more nuanced and adaptable feature extraction. This is complemented by an innovative sleep-cycle mechanism for periodic encoder retraining, ensuring continuous adaptation. Using two streams of natural images, we show that PCMC outperforms all baselines in both classification and clustering evaluation tasks. Additionally, ablation experiments explore the design choices behind PCMC, and showcase its memory utilization.

The experimental evaluation also shows that PCMC is not a perfect solution, and that it does not manage to completely avoid catastrophic forgetting. Consequently, we expect and hope that significant improvements will be feasible in the future. Follow-up work can also explore how to avoid the need for storing raw patches, potentially saving only patch embeddings in memory, and utilizing a decoder to restore the raw patches. Another improvement can focus on the sleep cycle, so that the learner sleeps when it needs to retrain its encoder and not periodically.

Acknowledgements This work was supported by the Lifelong Learning Machines (L2M) program of DARPA/MTO: Cooperative Agreement HR0011-18-2-0019.

REFERENCES

- Alessandro Achille, Tom Eccles, Loic Matthey, Chris Burgess, Nicholas Watters, Alexander Lerchner, and Irina Higgins. Life-long disentangled representation learning with cross-domain latent homologies. *Advances in Neural Information Processing Systems*, 31, 2018.
- Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 139–154, 2018.
- Rahaf Aljundi, Klaas Kelchtermans, and Tinne Tuytelaars. Task-free continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11254–11263, 2019a.
- Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. Gradient based sample selection for online continual learning. *Advances in neural information processing systems*, 32, 2019b.
- Guangji Bai, Qilong Zhao, Xiaoyang Jiang, Yifei Zhang, and Liang Zhao. Saliency-guided hidden associative replay for continual learning. *arXiv preprint arXiv:2310.04334*, 2023.

- Daniel Brignac, Niels Lobo, and Abhijit Mahalanobis. Improving replay sample selection and storage for less forgetting in continual learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 3540–3549, 2023.
- Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. *Advances in neural information processing systems*, 33:15920–15930, 2020.
- Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 132–149, 2018.
- Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Piotr Bojanowski, and Armand Joulin. Unsupervised learning of visual features by contrasting cluster assignments. *Advances in Neural Information Processing Systems*, 33: 9912–9924, 2020.
- Hyuntak Cha, Jaeho Lee, and Jinwoo Shin. Co2l: Contrastive continual learning. In *Proceedings of the IEEE/CVF International conference on computer vision*, pp. 9516–9525, 2021.
- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pp. 1597–1607. PMLR, 2020a.
- Xinlei Chen and Kaiming He. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 15750–15758, 2021.
- Xinlei Chen, Haoqi Fan, Ross Girshick, and Kaiming He. Improved baselines with momentum contrastive learning. *arXiv preprint arXiv:2003.04297*, 2020b.
- Giorgos Demosthenous and Vassilis Vassiliades. Continual learning on the edge with tensorflow lite. *arXiv preprint arXiv:2105.01946*, 2021.
- Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. Ieee, 2009.
- Aditya Deshpande, Jason Rock, and David Forsyth. Learning large-scale automatic image colorization. In *Proceedings of the IEEE international conference on computer vision*, pp. 567–575, 2015.
- Enrico Fini, Stéphane Lathuiliere, Enver Sangineto, Moin Nabi, and Elisa Ricci. Online continual learning under extreme memory constraints. In *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXVIII 16*, pp. 720–735. Springer, 2020.
- Enrico Fini, Victor G Turrissi Da Costa, Xavier Alameda-Pineda, Elisa Ricci, Karteek Alahari, and Julien Mairal. Self-supervised models are continual learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9621–9630, 2022.
- Yasir Ghunaim, Adel Bibi, Kumail Alhamoud, Motasem Alfarra, Hasan Abed Al Kader Hammoud, Ameya Prabhu, Philip HS Torr, and Bernard Ghanem. Real-time evaluation in online continual learning: A new hope. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11888–11897, 2023.
- Spyros Gidaris, Praveer Singh, and Nikos Komodakis. Unsupervised representation learning by predicting image rotations. *arXiv preprint arXiv:1803.07728*, 2018.
- Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. Bootstrap your own latent—a new approach to self-supervised learning. *Advances in neural information processing systems*, 33:21271–21284, 2020.
- Md Yousuf Harun, Jhair Gallardo, Tyler L Hayes, Ronald Kemker, and Christopher Kanan. Siesta: Efficient online continual learning with sleep. *arXiv preprint arXiv:2303.10725*, 2023.
- Tyler L Hayes and Christopher Kanan. Lifelong machine learning with deep streaming linear discriminant analysis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pp. 220–221, 2020.
- Tyler L Hayes and Christopher Kanan. Online continual learning for embedded devices. *arXiv preprint arXiv:2203.10681*, 2022.

- Tyler L Hayes, Kushal Kafle, Robik Shrestha, Manoj Acharya, and Christopher Kanan. Remind your neural network to prevent catastrophic forgetting. In *European Conference on Computer Vision*, pp. 466–483. Springer, 2020.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 9729–9738, 2020.
- Yen-Chang Hsu, Yen-Cheng Liu, Anita Ramasamy, and Zsolt Kira. Re-evaluating continual learning scenarios: A categorization and case for strong baselines. *arXiv preprint arXiv:1810.12488*, 2018.
- Julio Hurtado, Alain Raymond-Sáez, Vladimir Araujo, Vincenzo Lomonaco, Alvaro Soto, and Davide Bacciu. Memory population in continual learning via outlier elimination. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 3481–3490, 2023.
- Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. *Advances in neural information processing systems*, 33: 18661–18673, 2020.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. Learning representations for automatic colorization. In *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*, pp. 577–593. Springer, 2016.
- Ya Le and Xuan Yang. Tiny imagenet visual recognition challenge. *CS 231N*, 7(7):3, 2015.
- Byung Hyun Lee, Okchul Jung, Jonghyun Choi, and Se Young Chun. Online continual learning on hierarchical label expansion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 11761–11770, 2023.
- Soochan Lee, Junsoo Ha, Dongsu Zhang, and Gunhee Kim. A neural dirichlet process mixture model for task-free continual learning. *arXiv preprint arXiv:2001.00689*, 2020.
- Junnan Li, Pan Zhou, Caiming Xiong, and Steven CH Hoi. Prototypical contrastive learning of unsupervised representations. *arXiv preprint arXiv:2005.04966*, 2020.
- Zengyi Li, Yubei Chen, Yann LeCun, and Friedrich T Sommer. Neural manifold clustering and embedding. *arXiv preprint arXiv:2201.10000*, 2022a.
- Zhiyuan Li, Xiajun Jiang, Ryan Missel, Prashanna Kumar Gyawali, Nilesh Kumar, and Linwei Wang. Continual unsupervised disentangling of self-organizing representations. In *The Eleventh International Conference on Learning Representations*, 2022b.
- Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017.
- Zhiwei Lin, Yongtao Wang, and Hongxiang Lin. Continual contrastive learning for image classification. In *2022 IEEE International Conference on Multimedia and Expo (ICME)*, pp. 1–6. IEEE, 2022.
- David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. *Advances in neural information processing systems*, 30, 2017.
- Divyam Madaan, Jaehong Yoon, Yuanchun Li, Yunxin Liu, and Sung Ju Hwang. Representational continuity for unsupervised continual learning. *arXiv preprint arXiv:2110.06976*, 2021.
- Nicolas Michel, Romain Negrel, Giovanni Chierchia, and Jean-François Bercher. Domain-aware augmentations for unsupervised online general continual learning. *arXiv preprint arXiv:2309.06896*, 2023.

- Paweł Morawiecki, Andrii Kruttsylo, Maciej Wołczyk, and Marek Śmieja. Hebbian continual representation learning. *arXiv preprint arXiv:2207.04874*, 2022.
- Zixuan Ni, Siliang Tang, and Yueting Zhuang. Self-supervised class incremental learning. *arXiv preprint arXiv:2111.11208*, 2021.
- Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A Efros. Context encoders: Feature learning by inpainting. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2536–2544, 2016.
- Jason Ramapuram, Magda Gregorova, and Alexandros Kalousis. Lifelong generative modeling. *Neurocomputing*, 404:381–400, 2020.
- Amal Rannen, Rahaf Aljundi, Matthew B Blaschko, and Tinne Tuytelaars. Encoder based lifelong learning. In *Proceedings of the IEEE international conference on computer vision*, pp. 1320–1328, 2017.
- Dushyant Rao, Francesco Visin, Andrei Rusu, Razvan Pascanu, Yee Whye Teh, and Raia Hadsell. Continual unsupervised representation learning. *Advances in neural information processing systems*, 32, 2019.
- Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pp. 2001–2010, 2017.
- Mengye Ren, Tyler R Scott, Michael L Iuzzolino, Michael C Mozer, and Richard Zemel. Online unsupervised learning of visual representations and categories. *arXiv preprint arXiv:2109.05675*, 2021.
- David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. *Advances in Neural Information Processing Systems*, 32, 2019.
- Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. *Advances in neural information processing systems*, 30, 2017.
- James Smith, Cameron Taylor, Seth Baer, and Constantine Dovrolis. Unsupervised progressive learning and the stam architecture. *30th International Joint Conference on Artificial Intelligence*, 2021.
- Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pp. 1096–1103, 2008.
- Fei Ye and Adrian G Bors. Learning latent representations across multiple data domains using lifelong vaegan. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XX 16*, pp. 777–795. Springer, 2020.
- Xiaofan Yu, Yunhui Guo, Sicun Gao, and Tajana Rosing. Scale: Online self-supervised lifelong learning without prior knowledge. *arXiv preprint arXiv:2208.11266*, 2022.
- Richard Zhang, Phillip Isola, and Alexei A Efros. Colorful image colorization. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part III 14*, pp. 649–666. Springer, 2016.
- Song Zhang, Gehui Shen, Jinsong Huang, and Zhi-Hong Deng. Self-supervised learning aided class-incremental lifelong learning. *arXiv preprint arXiv:2006.05882*, 2020.
- Bolei Zhou, Agata Lapedriza, Aditya Khosla, Aude Oliva, and Antonio Torralba. Places: A 10 million image database for scene recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2017.

Table 3: PCMC Hyperparameters

Symbol	ImageNet-40	Places365-40	Description
patch size	60	60	the patch size used for patch extraction
stride	20	20	the stride used for patch extraction
pre-bs	256	256	the batch size used for pre-training
sleep-bs	512	512	the batch size used for sleep training
lr	0.6	0.6	the initial learning rate
wd	1e-5	1e-5	the weight decay
α	0.0	0.1	centroid learning rate
β	0.95	0.975	percentile for novelty detection distance threshold
ρ	0.25	0.15	used in definition of class informative centroids
Δ	400	400	the storage capacity of the STM
θ	30	30	number of matches for centroid copy to LTM
M	30	10	number of examples initially stored with each centroid
$M_{\min}$	5	5	minimum number of samples stored with each centroid
M_{init}	17	10	number of examples stored with the initial centroids
k	1	2	forgetting factor controlling example forgetting rate
K	1	1	number of examples utilized for centroid position update

Table 4: Augmentation Hyper-Parameters

Symbol	ImageNet-40	Places365-40
temperature	0.5	0.5
color jitter	0.8	0.8
brightness	0.6	0.6
contrast	0.6	0.6
saturation	0.6	0.6
hue	0.2	0.2
horizontal flip	0.5	0.5
vertical flip	0.0	0.0
random gray scale	0.2	0.2
Gaussian blur	0.2	0.2
kernel size	5	5
σ_1	0.1	0.1
σ_2	2.0	2.0
crop min	0.3	0.3
crop max	1.0	1.0

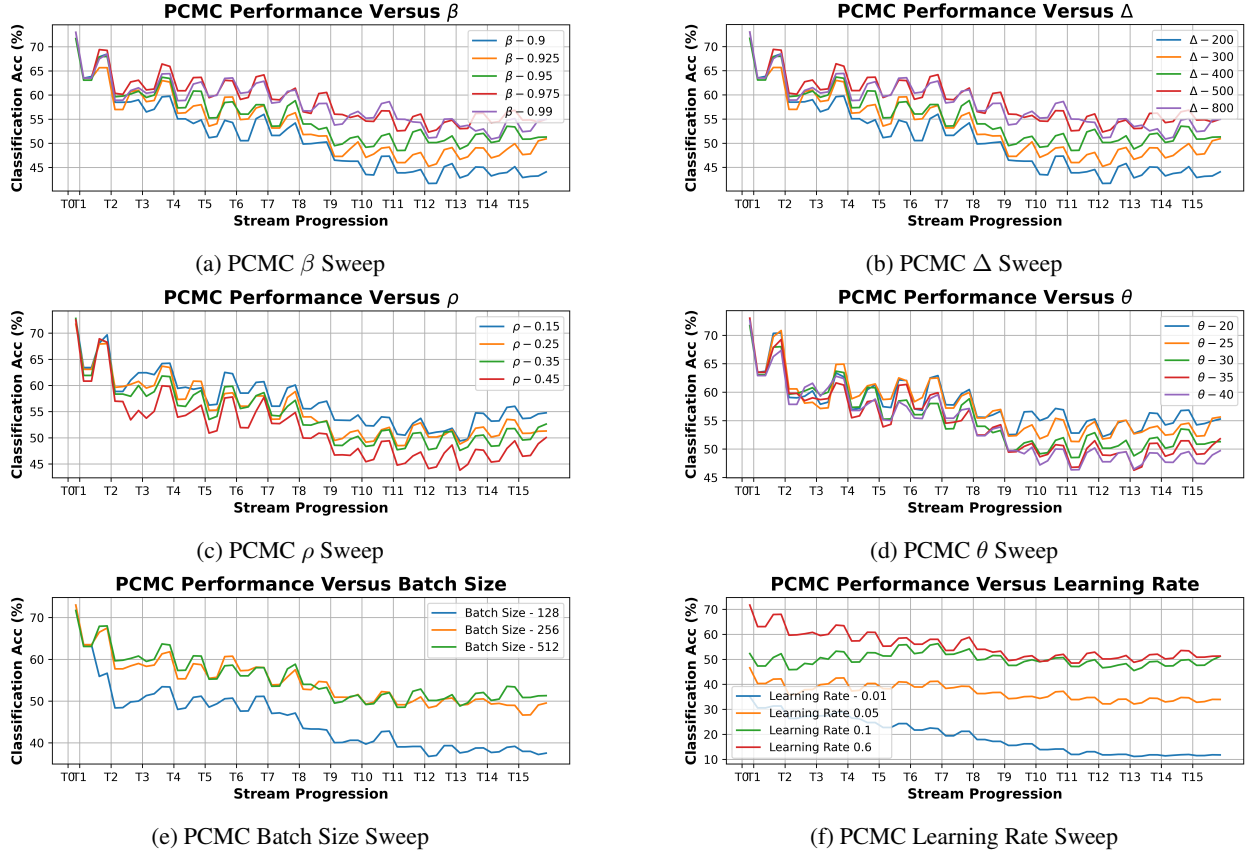


Figure 7: Hyper-parameter sweeps for a) β , b) Δ , c) ρ , d) θ , e) batch size, and f) learning rate for the PCMC model on the ImageNet-40 stream.

A ADDITIONAL PCMC DETAILS

A.1 MEMORY USAGE

PCMC does not have a direct cap on the total number of centroids and therefore also does not have an explicit cap on the number of raw patches that can be stored. However, the number of examples *per* centroid is controlled with the memory parameters M , $M_{\min}$, M_{init} , as well as the forgetting factor k in the memory consolidation step described in 2.3. Over the course of a each task PCMC tends to learn a consistent number of centroids given a fixed set of centroid learning hyper-parameters and so we can control the total memory usage per task fairly precisely. In our ImageNet-40 experiments, we set $M = 30$, $M_{init} = 17$ and $k = 1$ which provides us with 13600 patches initially in T0 and an average gain of ≈ 733 patches per task. Given the patch size of 60×60 this is equivalent to initially storing 3600 whole images and an average of 183 whole new images per streaming task (less than 10% of the available stream data).

We also consider the memory storage required for the centroids themselves. Since the latent space is in 512 dimensions we multiply the total number of floating point values by the total number of centroids $512 \times |LTM| \leq 2000$ in all experiments. This equates to an upper bound of 1,024,000 floating point values which is ≈ 24 images. We consider this negligible compared to the total memory requirement for storing patches itself and also allowed for the baselines memory capacity to be padded by at least 25 whole images for each experiment.

A.2 HYPER-PARAMETERS

In Table 3 we showcase all of the hyper-parameters utilized for our experiments with ImageNet-40, Places365-40 and the RGB-Objects streams, along with a description of each parameters purpose. In Table 4 we report the contrastive learning augmentations utilized for the contrastive pre and sleep training. In Figure 7 we explore changes to some of

Table 5: SCALE Hyper-Parameters

Symbol	ImageNet-40	Places365-40
lr	0.5	0.5
bs	0.5	0.5
temp	0.5	0.5
temp tsne	0.1	0.1
current temp	0.1	0.1
past temp	0.01	0.01
base temp	0.07	0.07
distill power	0.1	0.1
mem max classes	40	40
M	6500	6000
horizontal flip	0.5	0.5
crop min	0.08	0.08
crop max	1.0	1.0

PCMC’s key hyper-parameters and how this impacts the overall performance. Something to keep in mind for each of these results is that many of these parameters have an indirect impact on the total memory capacity of PCMC so when choosing our primary experiments hyper-parameters we weigh the overall performance with the total amount of memory utilized. For example, Subfigure 7a looks at the choice of β value which controls the sensitivity of the novelty detection. The highest performing values are 0.975 and 0.99 with 0.95 being close behind, however we end up with significantly more LTM centroids in the higher β cases and therefore, a larger memory capacity which is not justified by the relatively small performance gain. For the most part each choice of parameter is not that significant outside of the learning rate and batch size which must both be large enough (greater than 256 and 0.1 respectively).

B BASELINE DETAILS

B.1 SCALE

We adapt the SCALE code found [here](#) for the UPL setting. We make no changes to the loss or memory module, but add a wrapper which allows the model to interface with our streams. For clustering we utilize k-means with embeddings generated from the trained ResNet18 backbone with K equal to two times the number of classes. For classification we utilize KNN with K values of 1, 3, 5, and 50 (where 50 is what the original SCALE paper utilizes) and take the best performance per evaluation. The full set of hyper-parameters used can be found in Table 5.

In Figure 8 we demonstrate the performance of SCALE with varying batch sizes, memory sizes, temperatures, and learning rates. The major challenge for SCALE is learning quickly enough while also maintaining a large enough batch size. Because our streams are relatively short in terms of the total number of images per task, the batch size being large (> 256) starts to reduce the total number of parameter updates per task. For example, in the ImageNet-40 stream the total number of stream images per task is 2000, which means that with a batch size of 256 SCALE only gets ≈ 8 updates throughout the entire task. This leads to the performance preferring lowering the learning rate to maximize the performance from the offline pre-training. While SCALE wasn’t designed for our setting we wanted to include this modified version for a thorough analysis. We found that changes in the other key hyper-parameters did not have a significant impact on the performance.

B.2 WHOLE BASELINE

The whole baseline model is meant to serve as a simple approach, designed with similar ideas to the PCMC model, but significantly simplified. We compare with a similar set of hyper-parameters and optimization settings to the PCMC approach to ensure a fair comparison. For clustering we utilize KMeans with embeddings generated from the trained ResNet18 backbone with K equal to two times the number of classes. For classification we utilize KNN with K

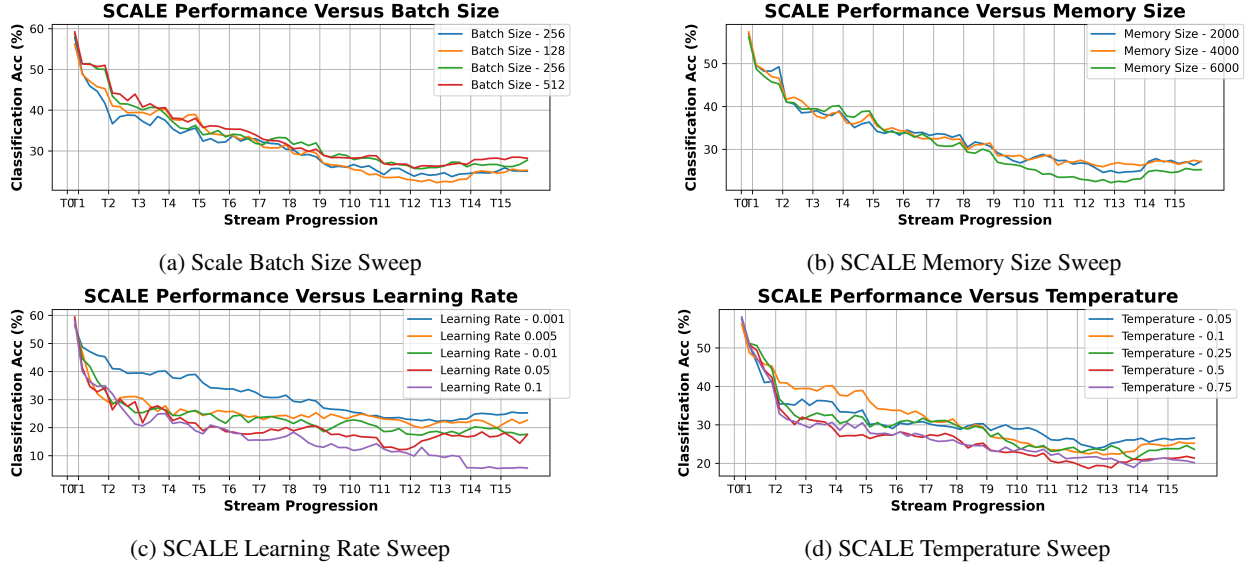


Figure 8: Hyper-parameter sweeps for a) batch size, b) memory size, c) learning rate, and d) temperature for the SCALE model on the ImageNet-40 stream.

Table 6: Whole Baseline Hyper-Parameters

Symbol	ImageNet-40	Places365-40
lr	0.5	0.5
bs	0.5	0.5
temp	0.5	0.5
color jitter	0.8	0.8
brightness	0.6	0.6
contrast	0.6	0.6
saturation	0.6	0.6
hue	0.2	0.2
horizontal flip	0.5	0.5
vertical flip	0.0	0.0
rand gray scale	0.2	0.2
Gaussian blur	0.2	0.2
kernel size	5	5
σ_1	0.1	0.1
σ_2	2.0	2.0
crop min	0.08	0.08
crop max	1.0	1.0
M	225	175
M_{init}	3750	2000

values of 1, 3, and 5 taking the best performance per evaluation. The full set of hyper-parameters used can be found in Table 6.

In Figure 9 we demonstrate how the whole baseline’s performance is impacted by changes in some of the key hyper-parameters including, learning rate, batch size, memory size, and temperature. For the most part the changes in these hyper-parameters did not have a significant impact on the models performance. We did find that having a larger batch size actually reduced performance slightly which was unexpected. We suspect that this is due to the limited memory size and therefore limited number of total iterations that are performed with the larger batch size. In Subfigure 9b we show how increasing the memory size does continue to improve performance, but in our primary experiments we utilized a memory size of 225 (per task) to reflect a closer upper bound to the total memory utilize for PCMC. When PCMC is allowed for increased memory usage it also has improved performance which out performs the whole baseline.

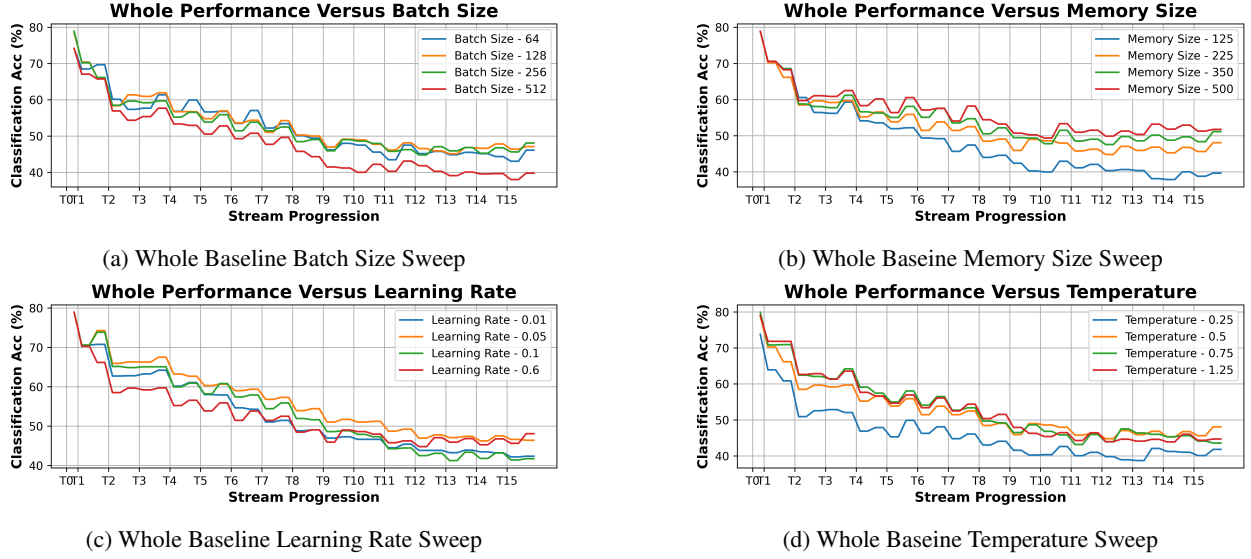


Figure 9: Hyper-parameter sweeps for a) batch size, b) memory size, c) learning rate, and d) temperature for the Whole Baseline model on the ImageNet-40 stream.

B.3 STAM

Table 7 showcases the hyper-parameters utilized for our experiments with STAM. We experimented with a few alternatives but found them to have little impact on the overall performance. Additionally, we intended to showcase the difference between our approach and STAM so we kept the parameters for our approach and STAM as similar as possible (while also accounting for the many differences in the overall approaches).

Table 7: STAM Hyper-parameters

Symbol	ImageNet-40	Places365-40
Λ	3	3
α	0.1	0.1
β	0.95	0.95
ρ	0.35	0.35
θ	30	30
Layer-0		
patch size	40	40
stride	20	20
Δ	1200	1200
Layer-1		
patch size	60	60
stride	30	30
Δ	800	800
Layer-2		
patch size	90	90
stride	30	30
Δ	400	400

C ADDITIONAL PCMC RESULTS

In this section we take a look at some additional plots which showcase different aspects of PCMC’s performance and learning.

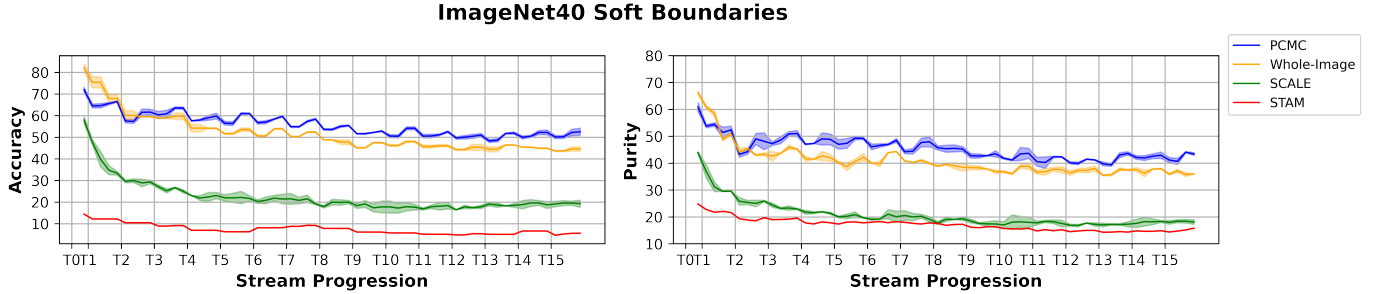


Figure 10: Classification and Clustering performance comparisons between PCMC and baselines on the ImageNet-40 and Places365-40 stream. In both streams the initial task T0 contains 10 classes and each of the subsequent 15 tasks contain 2 classes each. Each task contains four evaluations points spread evenly throughout the task and the classification and clustering tasks both focus on all classes seen so far. Task boundaries are blended to include increasing frequency of new task samples while previous task samples slowly fade. Classification utilizes 100 labeled examples per class and 100 test example per class for both tasks. Results calculated as the mean over three independent seeded trials with error measured as $\pm$ one standard deviation.

Table 8: Comparison of classification performance versus memory usage for PCMC with varying values of $M_{\min}$ and k . $|LTM|$ is the maximum memory (in terms of whole images) utilized and Accuracy represents the mean over all the evaluation points.

$M_{\min}$	$ LTM $	Accuracy
10	6713	57.3
15	7126	57.6
20	7760	57.5
k		
1	6046	56.3
2	7212	56.8
3	7310	56.9
4	7295	57.0

C.1 SOFT TASK BOUNDARIES

Figure 10 shows results from a similar experiment to the baseline comparison shown in Section 4. This experiment focuses on just ImageNet40, but with a modification to the task transitions. Instead of the traditional hard task boundaries typically considered in class incremental learning, we explore the performance of PCMC and the baselines on a stream where the boundary between tasks is soft. Classes from the new task are slowly introduced while the classes from the previous task slowly disappear. Specifically, we consider a blended region of 0.2 where the final 20% of the current task and the first 20% of the next task are blended together. Specifically we split this blended region into three equal partitions where the first partition consists of 25% new task data and 75% old task data, while the second partition consists of an even 50/50 split of both task data and the final partition is 75% new and 25% old task data.

Because PCMC and the baselines do not make any strong assumptions about the distribution of the data, we do not expect any significant decrease in performance by making this change. Looking at the results in Figure 10 we can see that indeed most of the methods perform very similarly to in the hard task boundary scenario. PCMC still has the best performance and the gap between it and the most competitive Whole-Image baseline is still roughly the same. This suggests that our method, as well as the baselines, are robust to small variations in the data stream and don't rely on hard task boundaries or long periods of fixed distributions to perform well.

C.2 MEMORY EXPLORATION

In this section we take a look at how the additional parameters $M_{\min}$ and k impact the memory size of PCMC over training. By increasing $M_{\min}$ the model must keep more examples stored with each LTM centroid. Additionally, the

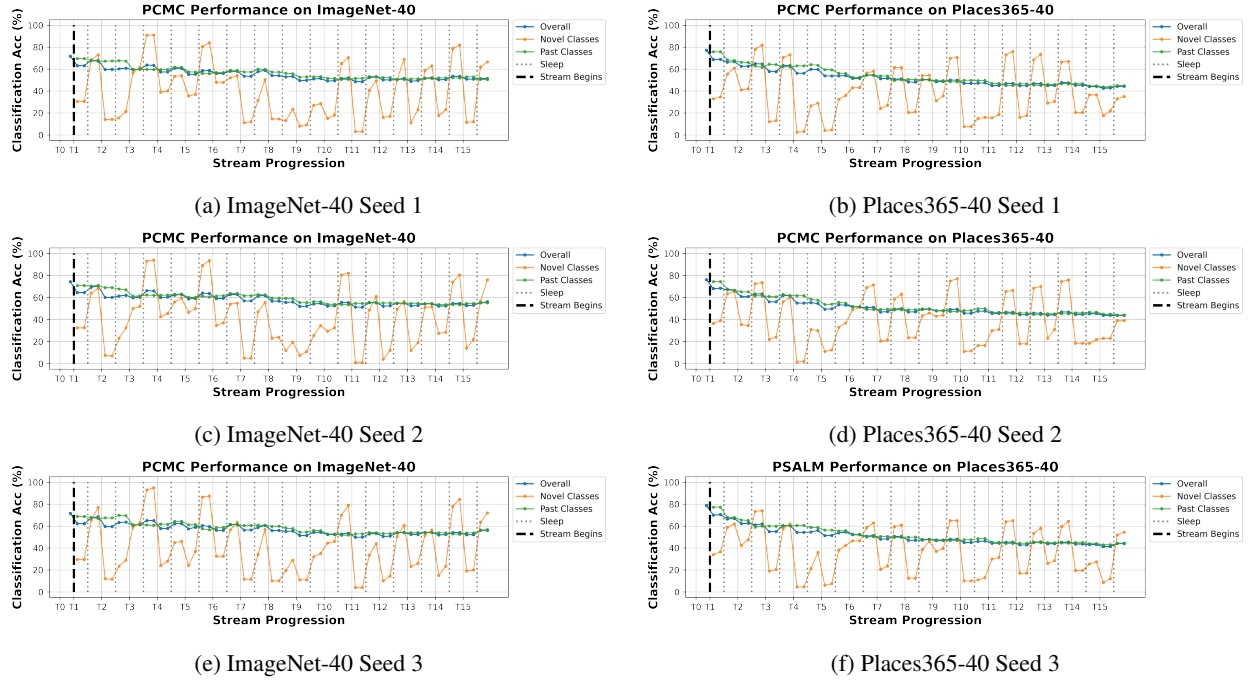


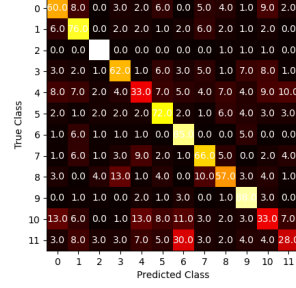
Figure 11: PCMC classification performance breakdown for three independent trials on the ImageNet-40 and Places365-40 streams. The red curve represents the performance on the novel classes, the blue curve represents performance on the past (previously observed) classes and the green curve represents the overall performance. The vertical grey dashed lines represent the sleep cycles.

probability given by Equation 5 is shifted higher as the curve slopes to 0 at $M_{\min}$. In the first three rows of Table 8 we see that increasing the minimum does slightly slightly increases the total memory storage, but has very little effect on the overall performance. This is because many centroids may contain redundant patches which are not useful for improving the encoder during training.

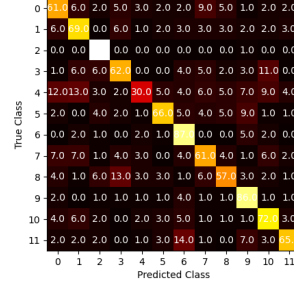
Similarly for the forgetting factor k , as we increase the value, the probability defined in Equation 5 becomes more convex and has a lower probability as the $|\mathcal{M}_j|$ gets smaller (relative to other values of k). This value may be particularly important in long streams where you want to control the total memory size over very large numbers of tasks (and sleeps), but you want the memory to decay very slowly when its becomes small. In the last three rows of Table 8, as k gets larger its relative impact decreases on both $|LTM|$, but also performance. We did not consider values of $0 < k < 1$ in this work, but depending on the needs to aggressively prune the memory, these values could produce much more drastic results.

C.3 INCORPORATING NEW CLASSES

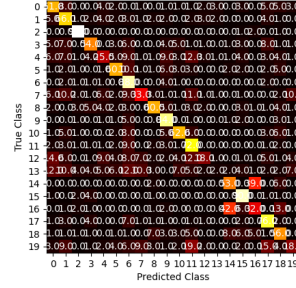
Figure 11 showcases additional seeds for the plots first introduced in Section 4.3. We can see that consistently, the sleep algorithm is able to increase the performance of the novel classes drastically. In Figure 12 we examine the class-wise performance pre and post sleep cycle for Tasks 1, 5, 10, and 15. For each task, the newest novel classes are represented by the bottom two rows/columns of the matrix. In each case the performance for those novel classes is fairly poor for the pre-sleep evaluation and improves significantly in the post-sleep evaluation. At the same time, we do not see a significant change in the overall performance for the other classes, indicating that the new knowledge has been successfully incorporated without forgetting.



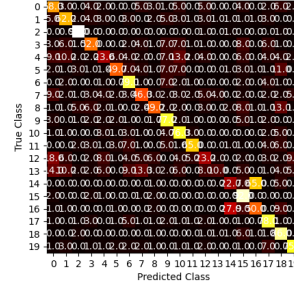
(a) T1 Confusion Matrix Before Sleep



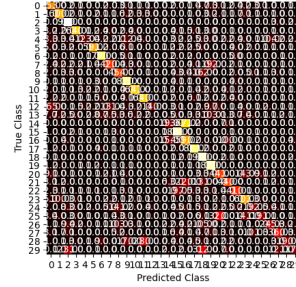
(b) T1 Confusion Matrix After Sleep



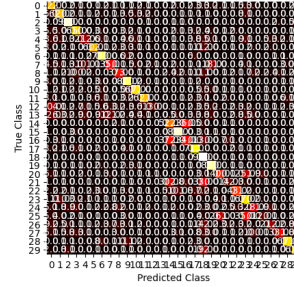
(c) T5 Confusion Matrix Before Sleep



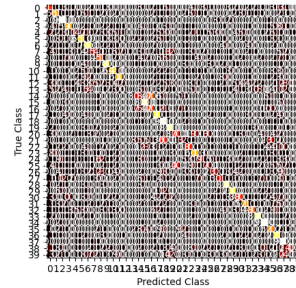
(d) T5 Confusion Matrix After Sleep



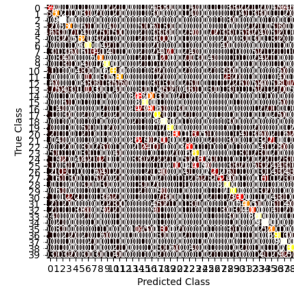
(e) T10 Confusion Matrix Before Sleep



(f) T10 Confusion Matrix After Sleep



(g) T15 Confusion Matrix Before Sleep



(h) T15 Confusion Matrix After Sleep

Figure 12: Visualization showing the classification confusion matrix for PCMC before and after the sleep cycle in tasks 1, 5, 10, and 15. Brighter colors indicate higher accuracy.